Speech Command Recognition with Reinforcement Learning-Optimized Threshold Tuning

Justine Luis Caluguas
School of Computing, Holy Angel University
Agapito del rosario Angeles City Pampanga
+639552037257
jscalaguas@student.hau.edu.ph

Charles Jansen Manusig
School of Computing, Holy Angel University
Manibaug Porac Pampanga
+639619030011
cvmanusig@student.hau.edu.ph

Carl Gaebriel Pagcu
School of Computing, Holy Angel University
Lagundi Mexico Pampanga
+639271339812
cjpagcu@student.hau.edu.ph

Justine Kevin Reyes
School of Computing, Holy Angel University
Sto. Domingo Angeles City Pampanga
+639291168300
jmreyes3@student.hau.edu.ph

1. Project and Dataset

The goal of this project is to build a speech command recognition system capable of classifying 35 spoken commands from audio input. The system is designed around a CNN trained on Mel spectrograms, with a Reinforcement Learning agent layered on top to tune the decision threshold under asymmetric error costs, specifically optimizing to minimize false rejections, where real commands are incorrectly discarded.

The primary dataset used is Google Speech Commands V2, sourced from Kaggle. The dataset contains approximately 105,000 one-second .wav recordings at 16kHz, covering 35 command classes spoken by over 2,000 unique speakers. Official train, validation, and test splits are provided via validation_list.txt and testing_list.txt, ensuring speaker-disjoint evaluation to reduce overfitting risk.

From the EDA, the dataset was found to be **largely balanced across classes** with minimal variance in sample counts per command. All audio clips were padded or trimmed to exactly 1 second (16,000 samples) before feature extraction.

2. Data Splits Finalized

The official splits from the dataset were used without modification:

1. Train: ~84,000 files (~80%)
2. Validation: ~9,900 files (~9.5%)
3. Test: ~11,000 files (~10.5%)

Speaker-disjoint splits ensure that no speaker appears in more than one split, which prevents the model from memorizing individual voice characteristics and provides a more realistic evaluation of generalization.

3. Baselines Trained

3.1 Simple ML Baseline - Dynamic Time Warping (DTW)

A DTW-based nearest-neighbor classifier was implemented as the simple ML baseline, following prior work on the Free Spoken Digit Dataset. Audio features were extracted as MFCCs (12 coefficients) using a custom filterbank pipeline. Classification was performed by finding the reference recording with the minimum DTW distance to the test sample. This approach achieved a Word Error Rate (WER) of approximately 5-10% on digit recognition, confirming it as a functional but limited baseline that does not scale well to 35-class command recognition.

3.2 Deep Learning Baseline - CNN on Mel Spectrograms

The primary model is a 4-block Convolutional Neural Network trained on log-Mel spectrograms (64 mel bins, 1024-pt FFT, hop length 512). The architecture consists of Conv2D → BatchNorm → ReLU → MaxPool blocks, followed by GlobalAveragePooling and a Dense classification head with softmax output across 35 classes.

Key training configuration:

4. Optimizer: Adam (lr=0.001, reduced on plateau)
5. Loss: Sparse categorical cross-entropy
6. Batch size: 64
7. Max epochs: 30 with early stopping (patience=5)
8. Best weights restored from epoch 27

4. CNN Training Results

The model was trained for 30 epochs. Early stopping identified epoch 27 as the best checkpoint based on validation accuracy. The learning rate was automatically reduced at epoch 30 via ReduceLROnPlateau. Selected epoch results are shown below:

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.4394	0.7235	1.9328	0.9133
5	0.8260	0.8981	0.5715	0.3556
10	0.8614	0.9236	0.4599	0.2625
15	0.8688	0.9256	0.4285	0.2463
27*	0.9114	0.9410	0.2844	0.1994
30	0.9120	0.9410	0.2823	0.1994

** Best epoch by validation accuracy — weights restored from this checkpoint.*

Training accuracy improved steadily from 43.9% at epoch 1 to 91.2% at epoch 30, with validation accuracy reaching a peak of 94.1%. The gap between training and validation accuracy remained small throughout, indicating the model generalized well without significant overfitting.

5. RL Agent - Threshold Tuning

A tabular Q-learning agent was implemented to tune the CNN's decision threshold after training. Rather than applying a fixed threshold of 0.50, the RL agent learns the optimal threshold by treating it as a discrete state space of 20 threshold values between 0 and 1.

The reward function is defined as the **negative expected cost**, computed using an asymmetric cost matrix where false rejects are penalized at 2.0 and false accepts at 1.0, reflecting the project goal of minimizing missed commands.

The agent was trained over 300 episodes on the validation set using epsilon-greedy exploration (epsilon=0.5, decay=0.995). The RL agent converged to an optimal threshold of 0.15, significantly lower than the default 0.50, reflecting the bias toward accepting more predictions to reduce false rejections.

6. Evaluation Metrics - Baseline vs. RL-Tuned

The table below compares the CNN evaluated at the default threshold (0.50) against the RL-tuned threshold (0.15) on the held-out test set:

Metric	Baseline (thr=0.50)	RL-Tuned (thr=0.15)	Change
Accuracy	0.9217	0.9373	+1.56%
Macro-F1	0.9598	0.9389	-2.09%
False Reject Rate	0.0439	0.0080	-81.8% ▼
False Accept Rate	0.0344	0.0547	+59.0%
Expected Cost	0.1222	0.0707	-42.1% ▼
Accept Rate	0.9561	0.9920	+3.75%

Key finding: The RL-tuned threshold reduced the false reject rate by 81.8% (from 4.39% to 0.80%) and reduced expected cost by 42.1% (from 0.1222 to 0.0707). Overall accuracy improved by 1.56 percentage points. The trade-off is a slight increase in false accepts (+2.03%) and a marginal decrease in macro-F1, which is acceptable given the project objective of minimizing missed commands.

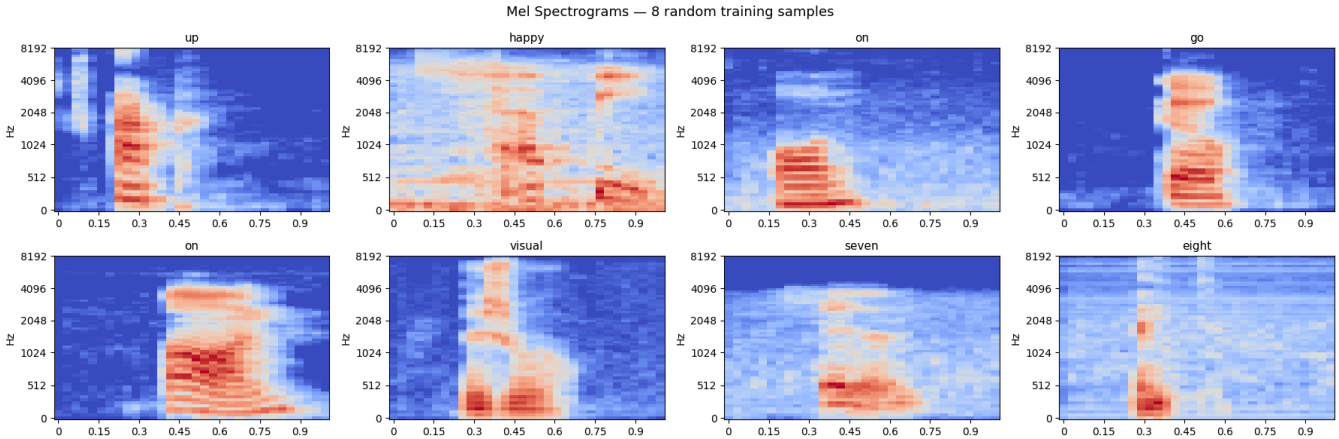
7. Ethics Statement - Fairness Audit

A per-speaker fairness audit was conducted on the test set across 224 speakers with 10 or more test samples. The audit evaluated per-speaker accuracy and false reject rate to identify potential bias across speakers, which may correlate with accents, dialects, or recording conditions.

Results: The mean per-speaker accuracy was 0.925 and the mean false reject rate was 0.014. However, the distribution revealed notable outliers: some speakers achieved accuracy as low as 0.125, with false reject rates reaching 0.906. This disparity suggests the model may underperform for speakers with less common accents or recording conditions not well-represented in the training data.

These findings highlight the need for accent-aware data augmentation and potentially speaker-adaptive thresholding in future iterations. Consent for audio usage is covered under the Google Speech Commands V2 dataset license, which was collected under informed consent protocols.

Initial Performance Metrics Pictures



Layer (type)	Output Shape	Param #
mel_spectrogram (InputLayer)	(None, 64, 32, 1)	0
conv2d_4 (Conv2D)	(None, 64, 32, 32)	288
batch_normalization_4 (BatchNormalization)	(None, 64, 32, 32)	128
activation_4 (Activation)	(None, 64, 32, 32)	0
max_pooling2d_4 (MaxPooling2D)	(None, 32, 16, 32)	0
dropout_5 (Dropout)	(None, 32, 16, 32)	0
conv2d_5 (Conv2D)	(None, 32, 16, 64)	18,432
batch_normalization_5 (BatchNormalization)	(None, 32, 16, 64)	256
activation_5 (Activation)	(None, 32, 16, 64)	0
max_pooling2d_5 (MaxPooling2D)	(None, 16, 8, 64)	0
dropout_6 (Dropout)	(None, 16, 8, 64)	0
conv2d_6 (Conv2D)	(None, 16, 8, 128)	73,728
batch_normalization_6 (BatchNormalization)	(None, 16, 8, 128)	512
activation_6 (Activation)	(None, 16, 8, 128)	0
max_pooling2d_6 (MaxPooling2D)	(None, 8, 4, 128)	0
dropout_7 (Dropout)	(None, 8, 4, 128)	0
conv2d_7 (Conv2D)	(None, 8, 4, 128)	147,456
batch_normalization_7 (BatchNormalization)	(None, 8, 4, 128)	512

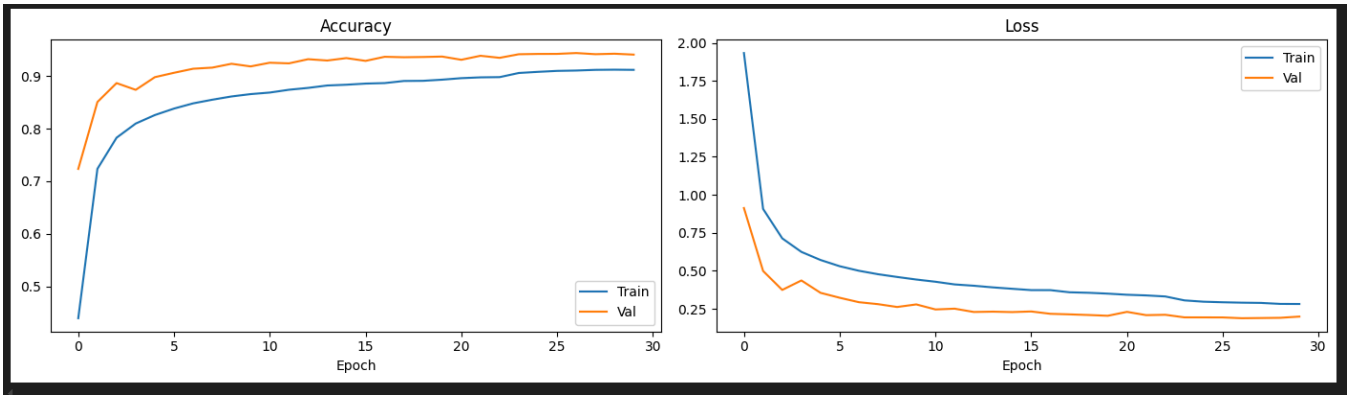
activation_7 (Activation)	(None, 8, 4, 128)	0
max_pooling2d_7 (MaxPooling2D)	(None, 4, 2, 128)	0
dropout_8 (Dropout)	(None, 4, 2, 128)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	0
dense_1 (Dense)	(None, 256)	33,024
dropout_9 (Dropout)	(None, 256)	0
probs (Dense)	(None, 35)	8,995

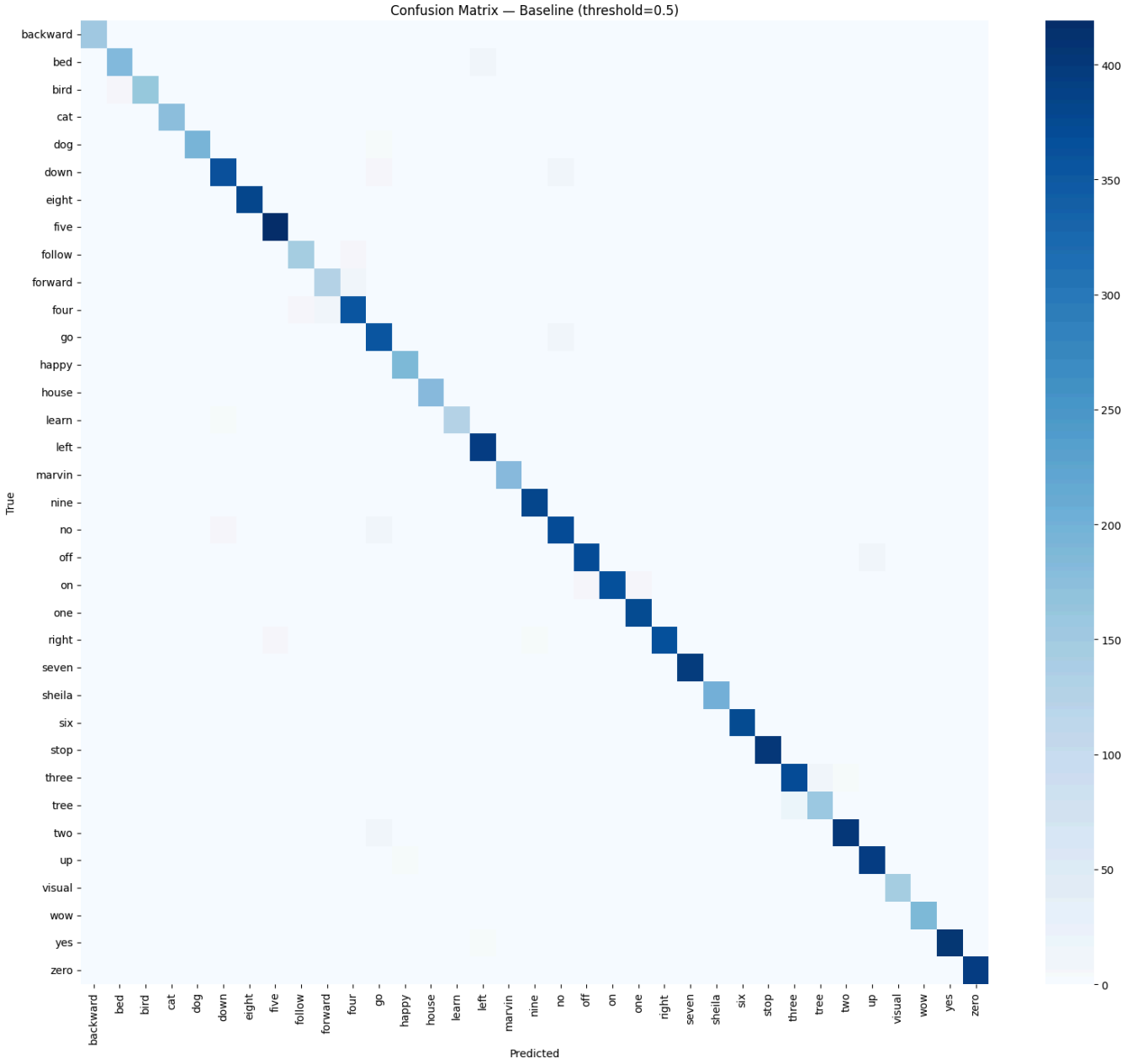
Total params: 283,331 (1.08 MB)

Trainable params: 282,627 (1.08 MB)

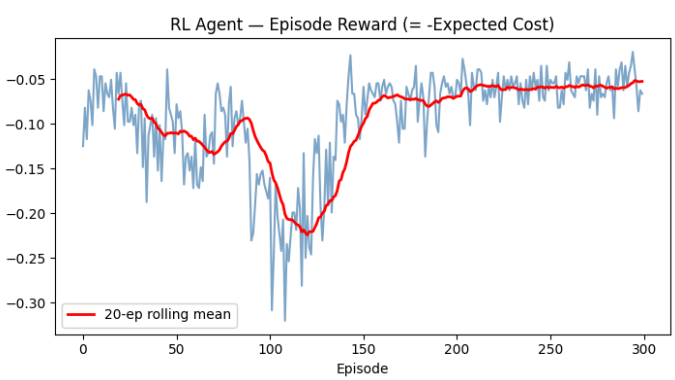
Non-trainable params: 704 (2.75 KB)

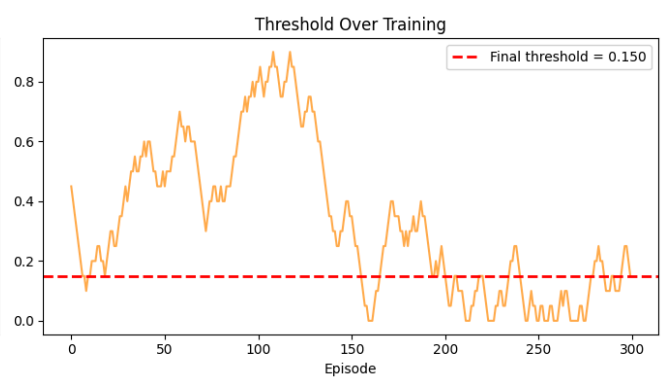
```
Epoch 1/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 46s 33ms/step - accuracy: 0.4394 - loss: 1.9328 - val_accuracy: 0.7235 - val_loss: 0.9133 - learning_rate: 0.0010
Epoch 2/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.7237 - loss: 0.9075 - val_accuracy: 0.8508 - val_loss: 0.4994 - learning_rate: 0.0010
Epoch 3/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.7829 - loss: 0.7138 - val_accuracy: 0.8868 - val_loss: 0.3747 - learning_rate: 0.0010
Epoch 4/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.8098 - loss: 0.6248 - val_accuracy: 0.8740 - val_loss: 0.4369 - learning_rate: 0.0010
Epoch 5/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.8260 - loss: 0.5715 - val_accuracy: 0.8981 - val_loss: 0.3556 - learning_rate: 0.0010
Epoch 6/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 42s 31ms/step - accuracy: 0.8383 - loss: 0.5306 - val_accuracy: 0.9064 - val_loss: 0.3231 - learning_rate: 0.0010
Epoch 7/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.8483 - loss: 0.5008 - val_accuracy: 0.9141 - val_loss: 0.2940 - learning_rate: 0.0010
Epoch 8/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 42s 32ms/step - accuracy: 0.8552 - loss: 0.4780 - val_accuracy: 0.9162 - val_loss: 0.2811 - learning_rate: 0.0010
Epoch 9/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 42s 32ms/step - accuracy: 0.8614 - loss: 0.4599 - val_accuracy: 0.9236 - val_loss: 0.2625 - learning_rate: 0.0010
Epoch 10/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 42s 32ms/step - accuracy: 0.8659 - loss: 0.4430 - val_accuracy: 0.9185 - val_loss: 0.2794 - learning_rate: 0.0010
Epoch 11/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.8688 - loss: 0.4285 - val_accuracy: 0.9256 - val_loss: 0.2463 - learning_rate: 0.0010
Epoch 12/30
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 43s 32ms/step - accuracy: 0.8741 - loss: 0.4109 - val_accuracy: 0.9244 - val_loss: 0.2514 - learning_rate: 0.0010
Epoch 13/30
...
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9114 - loss: 0.2844
Epoch 30: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
1326/1326 ━━━━━━━━━━━━━━━━━━━━ 42s 32ms/step - accuracy: 0.9120 - loss: 0.2823 - val_accuracy: 0.9410 - val_loss: 0.1994 - learning_rate: 5.0000e-04
Restoring model weights from the end of the best epoch: 27.
```

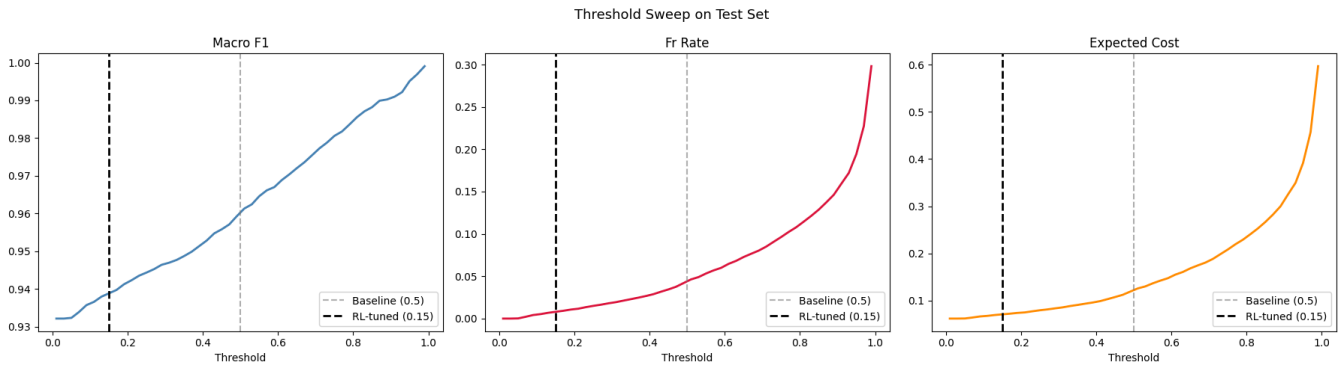



Confusion Matrix — Baseline (threshold=0.5)
True
Predicted
backward
bed
bird
cat
dog
down
eight
five
follow
forward
four
go
happy
house
learn
left
marvin
nine
no
off
on
one
right
seven
sheila
six
stop
three
tree
two
up
visual
wow
yes
zero
0
50
100
150
200
250
300
350
400

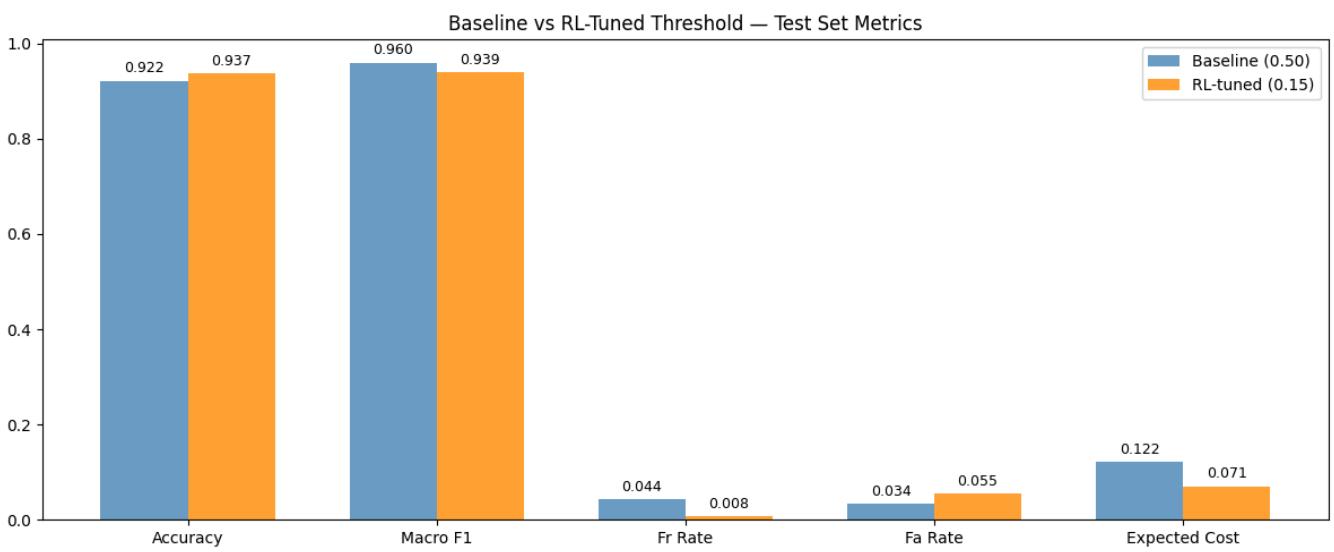
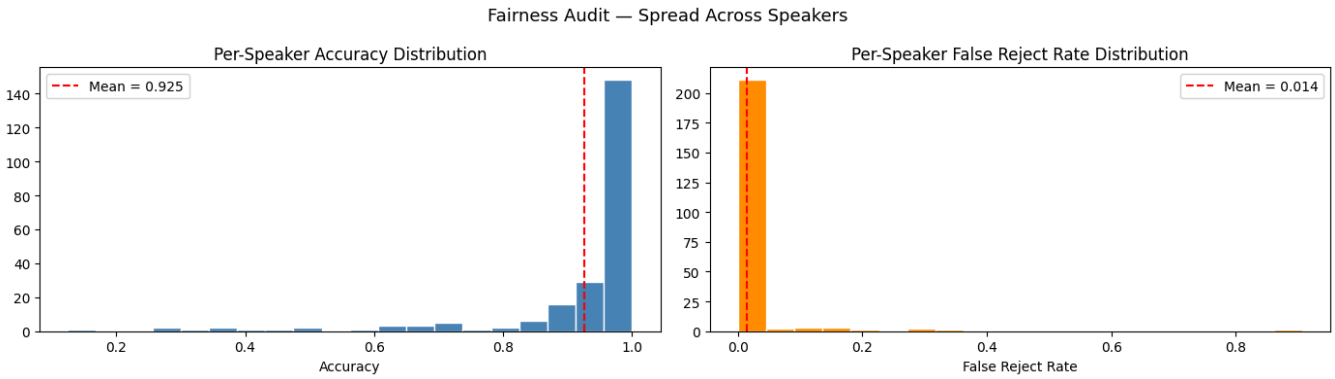
	precision	recall	f1-score	support
backward	0.98	0.98	0.98	158
bed	0.90	0.93	0.92	197
bird	0.98	0.93	0.96	173
cat	0.97	0.97	0.97	180
dog	0.95	0.95	0.95	206
down	0.95	0.94	0.95	385
eight	0.98	0.98	0.98	389
five	0.97	0.98	0.97	429
follow	0.92	0.89	0.90	162
forward	0.92	0.90	0.91	143
four	0.94	0.96	0.95	368
go	0.92	0.94	0.93	377
happy	0.97	0.97	0.97	194
house	0.99	0.97	0.98	186
learn	0.89	0.89	0.89	143
left	0.95	0.98	0.97	401
marvin	1.00	0.97	0.99	186
nine	0.97	0.97	0.97	391
no	0.94	0.96	0.95	387
off	0.98	0.97	0.97	384
on	0.96	0.96	0.96	380
one	0.96	0.98	0.97	384
right	0.99	0.96	0.97	382
...				
accuracy			0.96	10522
macro avg	0.96	0.96	0.96	10522
weighted avg	0.96	0.96	0.96	10522








```
── Test Set Results ──────────────────────────────────────────
                 Baseline (thr=0.50)  RL-tuned (thr=0.15)
accuracy                      0.9217               0.9373
macro_f1                      0.9598               0.9389
fr_rate                       0.0439               0.0080
fa_rate                       0.0344               0.0547
expected_cost                 0.1222               0.0707
accept_rate                   0.9561               0.9920
```



```
Speakers with >=10 test samples: 224
          n_samples  accuracy   fr_rate
speaker
8ea6dec6         32  0.125000  0.906250
3659fc1c         34  0.382353  0.323529
475b61f1         23  0.260870  0.304348
a9f54d8d         14  0.357143  0.285714
dfdabe19         28  0.607143  0.214286
220ee1ef         17  0.294118  0.176471
82d0d3ba         18  0.611111  0.166667
c9e251d2         30  0.500000  0.166667
2005ca25         27  0.481481  0.111111
f17d21b5         11  0.727273  0.090909
```